

FuzzFabric: A Scalable Grey-box Fuzzing Framework for Polymorphic Network Services

Bo Yu¹, Lei Zhou^{1*}, Chengnuo Cai¹, Qiang Yang¹, Yaojia Yang¹, Runhao Liu¹
Lei Luo¹, Wenyu Long¹, Lianghua Gong²

¹College of Computer Science and Technology, National University of Defense Technology

²Fengtai Technology (Beijing) Co., Ltd.

Email: {yubo0615, zhoulcs, caichengnuo277, q.yang, yangyaojia, runhaoliu, longwenyu}@nudt.edu.cn
luolei23@163.com, gonglianghua@fengtaisecl.com

Abstract—Fuzzing network protocols is challenging due to their various factors including syntax, state transition, and even conformation of its executable software. To achieve better convenience of platform development and effectiveness of individual protocol testing, current grey-box fuzzing approaches generally focus on limited factors. However, it leaves poor performance on real-world services's protocol evaluation since it requires significant additional adaptation efforts to accommodate the diversity of protocol software. To address this issue, we present FuzzFabric, a scalable grey-box fuzzing framework designed for polymorphic protocol services. FuzzFabric decouples existing fuzzing engines and modular programs the protocol processing adapters, and finally adaptive assembles fuzzing engine to evaluate different network protocol services. We implemented FuzzFabric on top of AFLNet, supporting 3 different protocol types and 4 instrumentation tools, and conducted experiments using 11 real-world protocol applications. The results demonstrate that FuzzFabric can effectively fuzz protocols in different forms with various instrumentation, validating its practical efficacy and scalability.

Index Terms—Grey-box fuzz, Network protocol, Polymorphic software, Instrumentation

I. INTRODUCTION

In today's interconnected world, network services play a crucial role in enabling communication between devices. However, the presence of weaknesses or vulnerabilities in network protocol applications can expose connected parties to security risks and potentially compromise the overall security of the network. For example, Microsoft officially disclosed and addressed a remote code execution vulnerability (CVE-2022-21907) in the Microsoft Windows HTTP protocol stack [1]. This vulnerability can be exploited without the need for identity authentication or user interaction. These occurrences necessitate comprehensive scrutiny of network protocol software, particularly network servers, as they are often exposed to the internet. Therefore, the identification and mitigation of these known or unknown vulnerabilities are crucial and pressing.

It is essential to carefully analyze and address these vulnerabilities to ensure the integrity and safety of network communications. Implementing robust security measures and

staying informed about potential threats is key to mitigating these risks. The use of the fuzzing technique has been proven to be highly effective in discovering vulnerabilities. However, fuzzing network services' protocols pose distinct and challenging differences compared to fuzzing command-line input binaries, as they frequently involve unique state spaces and complex interactions between clients and servers.

Recent researches focus on protocol fuzzing started from black-box methods based on expertise and predefined templates [2]–[6]. Presently, researchers pivot to grey-box fuzzing techniques for the same purpose [7]–[12]. These works incorporate protocol states into classic grey-box fuzzing engines to achieve both state and code coverage and experiment on specified protocols for effectiveness. As listed in Table I according to their implementations, each is restricted to specific protocol forms in terms of instrumentation.

We note that many existing network protocol fuzzers require testers to provide a specific protocol specification, which means that fuzzers should implement specific operations to satisfy the fuzzing preparing and result analyzing stage. For instance, AmpFuzz [13] integrates with a static analyzer to extract the control-flow graph (CFG) of the target protocol, which is an extra pre-processing work to prioritize the fuzzing inputs and improve UDP-type protocol fuzzing performance. Further investigation reveals that the supported protocol form is a crucial component of the kernel fuzzing engine in these proposed mechanisms. Considering that real-world protocol software is often polymorphic, such as the open-source OpenSSL, which can be used as a standalone application or customized and embedded into firmware, requiring additional adaptation efforts when implementing them in fuzzing practices [14].

Considering the above challenges, we present a scalable architecture for a grey-box fuzzing framework designed for network services' protocol. This framework introduces an abstract layer called the ProtoAgent, which serves to separate the fuzzing kernel from the tested protocol targets. Additionally, it constructs the fuzzing engine using protocol software of various forms to accommodate practical needs. The ProtoAgent offers uniform APIs to both sides and necessitates minor

*is the Corresponding Author.

modifications to existing mechanisms. As a prototype, we have implemented the framework on top of AFLNet and named it FuzzFabric.

In summary, our main contributions are as follows:

- We employ a loose-coupling design methodology to enhance existing grey-box fuzzing frameworks with support for polymorphic protocol software. We develop an abstract layer that serves to bridge the gap between the fuzzing engine and diverse protocol targets.
- We designed the ProtoAgent and integrated it into the AFLNet to implement the FuzzFabric, and subsequently prototyped the FuzzFabric with 3 typical protocol forms and 4 instrumentation tools. The prototype of FuzzFabric will be open-sourced at <https://github.com/ipasslab/fuzzfabric>.
- We evaluate the FuzzFabric using 11 real-world network protocol applications to validate the proposed solution towards a scalable framework.

The remainder of this paper is structured as Section II re-emphasizes the problems we focus on as well as further analyses. Section III introduces the FuzzFabric framework, primarily the ProtoAgent and the implemented protocol software forms in terms of instrumenting environments. We evaluate the framework in Section IV and explore related research in Section V. Section VI concludes this paper.

II. PROBLEM AND DISCUSSION

Grey-box network protocol fuzzing approaches are developed based on classic grey-box program fuzzers, such as AFL (American Fuzzy Lop) [15], and involve significant adaptation work, including: (1) creation of additional network client for protocol session interaction; (2) generation of fuzzing seeds by leveraging the special protocol parser and the message sequences of the network session; and (3) mutation of fuzzing seeds with consideration of communication state and protocol code coverage factors, which is usually based on target instrumentation. The study demonstrates effectiveness in testing specified protocol application examples. However, our investigations into related implementations reveal that directly testing polymorphic protocol software using existing frameworks is challenging. Significant additional work is necessary to modify the fuzzing engine to adapt to a new form of target and its instrumentation environment.

In our analysis, we categorize real-world network services' protocols into three types based on their execution and instrumentation restrictions. These types are: *Independent Open-Source Software (Type-I)*, *Standalone Executable Binary Program (Type-II)*, and *Embedded Code Component (Type-III)*. The Type-I is favored for fuzzing work due to its clear code and data semantics, as well as structured descriptions. The Type-II can generally be well executed and instrumented with advanced platforms like Qemu [16]. The Type-III encompasses many protocol applications that are tightly coupled with integrated systems, such as protocols functioning as modules in an OS kernel or executable programs relying on other functions. Instrumenting Type-III protocols without constructing a

TABLE I
PROTOCOL ADAPTION OF EXISTING GREY-BOX FUZZING RESEARCH

Fuzzer	Adaption protocols	Protocol Type
AFLNet [7]	DAAP\DICOM\DNS\DTLS\FTP\RTSP\SIP\SMTP\SSH\TLS	I*
StateAFL [8]	DAAP\DICOM\DNS\DTLS\FTP\RTSP\SIP\SMTP\SSH\TLS	I
SNAPFuzz [9]	DAAP\DICOM\DNS\DTLS\FTP\RTSP\SIP\SMTP\SSH\TLS	I
FIRM-AFL [10]	HTTP\DNS\SSH	III
SNPSFuzzer [11]	DTLS\SIP	II
Fw-Fuzz [12]	HTTP\SSH\FTP\RTSP	III
IoTHunter [20]	FTP\SMB\SNMP	III

* The **Type-II** support of AFLNet is limited and incomplete according to its code in the open-source repository [21]. Protocol applications cannot properly execute due to the old Qemu version employed.

full execution environment is difficult. Currently, only a few fuzzing approaches are designed for the Type-III, relying on optimized Qemu, PANDA [17], Dynamips [18], etc.

Table I lists supported protocol forms of existing studies according to the above classification. AFLNet [7], StateAFL [8] and SNAPFuzz [9] use Profuzzbench [19] to benchmark their work, but all are only compatible with **Type-I** protocols. All require the fuzzing engine to add specific code to accommodate new protocol forms. FirmAFL [10], Fw-Fuzz [12] and IoTHunter [20] manage to fuzz protocols through full-system simulation or real execution, but the fuzz kernel is tightly coupled with instrumentation environment. Considering increasing protocol forms and instrumentation methods, this is unscalable and time-expensive to meet diverse real-world fuzzing requirements.

Therefore, beyond assessing the functional completion of the fuzzing approach for a particular network protocol, users must undertake significant work to adapt appropriate pre-processing operations and the fuzzing framework to diverse target network protocols. It is essential to separate all auxiliary but necessary operations from both protocol preprocessing and fuzzing testing. The objective of FuzzFabric is to accommodate the variations among protocols and offer modular functional units for diverse fuzzing platforms.

III. FUZZFABRIC

In this section, we introduce FuzzFabric, and provide an overview of the framework in Figure 1. FuzzFabric adopts a modular software design to facilitate loose coupling and scalable fuzzy preprocessing and testing.

A. Architecture

The FuzzFabric modularizes three main components, inheriting the kernel's state-coverage-guided mechanism from AFLNet, as depicted in the right dotted rectangle of Figure 1. The key component of our framework is the middleware named ProtoAgent, situated between the protocol pre-process

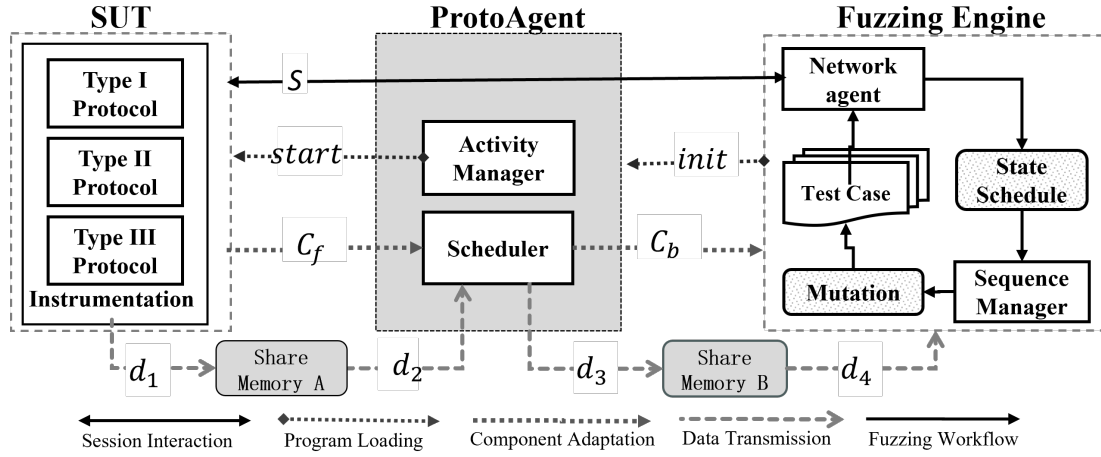


Fig. 1. General structure of the FuzzFabric framework

interface and the fuzzing engine. All other components and functions that directly interact with or are tightly coupled with the tested target (Service Under Testing, SUT) are shifted to or integrated into the ProtoAgent in the middle-dotted rectangle, for example, the network client. This design allows the fuzzing engine of FuzzFabric to focus on providing interesting test cases and interact with the ProtoAgent, aligning with previous discussions on scalability issues. The SUT represents all three protocol application forms supported by the framework.

The design of FuzzFabric emphasizes the issues highlighted in Section II and underscores the significance of the ProtoAgent. In the context of FuzzFabric, the ProtoAgent acts as a crucial intermediary between the fuzzing engine and the tested targets. It serves the dual purpose of receiving test cases (packet sequences) from the fuzzing engine and collecting feedback from the target, subsequently redirecting this feedback to the fuzzing engine. As illustrated in Figure 1, the ProtoAgent component incorporates two primary interaction modules: *Scheduler* and *Activity Manager*. The former manages the testing process, including functions like execution path feedback and execution status feedback, and also acts as a middleware facilitating communication between the fuzzing engine and the SUT, thereby coordinating the entire fuzzing procedure. The latter is responsible for initializing the SUT and creating shared memory modules A and B when the AFLnet launches the Agent, ensuring a smooth sharing of fuzzing inputs with the fuzzing engine.

B. Workflow of FuzzFabric Execution

In order to clarify the orchestration between different modules of the framework, particularly the ProtoAgent, we abstract one complete round of a fuzzing test into 4 steps (S1-S4) as follows:

- **S1:** Beginning (denoted as $\mathbb{B}\mathbb{E}$) phase, which maintains the session interaction S between SUT and fuzzing engine, meanwhile adapt right protocol parser and fuzzing engine with C_f and C_b operations.

- **S2:** Fuzzing (denoted as $\mathbb{F}\mathbb{Z}$) phase, after transferring special test cases through d_1 to d_4 steps, the fuzzing engine execute of protocol functions under the prepared execution environment and parameters.
- **S3:** Ending (denoted as $\mathbb{E}\mathbb{D}$) phase, which involves one-time instructions, including saving the state of the target protocol execution and rolling back to the previous state.
- **S4:** Feedbacking (denoted as $\mathbb{F}\mathbb{B}$) phase, focusing on fuzzing engine mutation, which includes new seed generation and preparation for a new round of testing with re-init and start SUT.

Meanwhile, we abstract the following operations to simplify the definition of interfaces for module interactions when testing a target (T):

- $Wrt(mem, data)$: write data into memory.
- $Read(mem, data)$: read data from memory.
- $Start(T)$: start a target for the test. .
- $Send(T, data)$: send data to a target.
- $Rcv(T, data)$: receive data from a target.
- $Com(T, in)$: communicate with a target to feed input.
- $Kill(T)$: terminate a tested target.
- $After(E1, E2, E2)$: a series of events E_x occur in the time order.
- Conditional $Ready(T)$: if the target is ready.
- Conditional $Exit(T)$: if the target is successfully closed.

Given a protocol application $\mathbf{P}$ instrumented by tools $\mathbf{I}$, the fuzzing engine $\mathbf{F}$ tests $\mathbf{P}$ and gets feedback $\mathbf{FI}$ from the requested memory block $\mathbf{MEM}$. By introducing the ProtoAgent

as **PA**, the fuzzing round can be expressed as:

$$\begin{aligned}
\mathbb{BE} &= \mathbf{PA.Wrt}(\mathbf{MEM}, 0) \wedge \mathbf{PA.Start}(\mathbf{P}) \\
&\quad \wedge \mathbf{PA.Ready}(\mathbf{P}) \\
\mathbb{FZ} &= \mathbf{F.Send}(\mathbf{PA}, \text{input}) \wedge \mathbf{PA.Rcv}(\mathbf{F}, \text{input}) \\
&\quad \wedge \mathbf{PA.Com}(\mathbf{P}, \text{input}) \wedge \mathbf{I.Write}(\mathbf{MEM}, \mathbf{FI}) \\
\mathbb{ED} &= \mathbf{PA.Exit}(\mathbf{P}) \vee \mathbf{F.Kill}(\mathbf{P}) \wedge \mathbf{F.Exit}(\mathbf{P}) \\
\mathbb{FB} &= \mathbf{After}(\mathbb{BE}, \mathbb{FZ}, \mathbb{ED}) \wedge \mathbf{PA.Read}(\mathbf{MEM}, \mathbf{FI}) \\
&\quad \wedge \mathbf{PA.Send}(\mathbf{F}, \mathbf{FI}) \wedge \mathbf{F.Rcv}(\mathbf{PA}, \mathbf{FI})
\end{aligned}$$

The cyclic fuzzing flow is scheduled according to the outlined procedures in Algorithm 1. When juxtaposed with Figure 1, the Activity Manager assumes responsibility for initiating the target (*Wrt* operation), monitoring its state (*Kill* and *After* operation), and other related tasks. This parallels the partial role played by the engine in existing fuzzing frameworks, albeit we position it as the sole entity for accommodating adaptations to various protocols through specific APIs. The scheduler oversees the fuzzing phase in accordance with Algorithm 1. It functions as an intermediary to intercept signals between the fuzzing engine and the SUT. For example, it may intercept signals such as the target termination signal emitted by the fuzzing engine, then transform it to detect the status of the working process. Subsequently, the ProtoAgent interfaces with the fuzzing engine, remaining mostly intact in its original state.

Algorithm 1: The fuzzing procedure of the FuzzFabric

Data: the SUT P , the input message
Result: Information of crashes

```

1 initialization;
2 while time not expire do
3   Execute  $\mathbb{BE}$ ;
4   repeat
5      $F$  performs mutation to generate new test case;
6     Execute  $\mathbb{FZ}$ ;
7     Execute  $\mathbb{ED}$ ;
8     Execute  $\mathbb{FB}$ ;
9   until  $P$  is crashed;
10  Store the information of crash
11 end

```

C. Instrumentation

To prototype and facilitate using the proposed FuzzFabric framework, we provide instrumenting environments to adapt to different protocol forms, namely Type-I, II, and III as discussed in Section II. Since grey-box fuzzing frameworks normally collect feedback in an edge-covered form, the FuzzFabric follows this mechanism but eliminates the fork-server burden imposed on instrumentations. It enriches the supported protocol forms as:

For Type-I: Because *llvm* [22] is sufficient to instrument open-source software developed in languages like Objective-C, C, and C++, no new tools are introduced. The instrumentation

method is inherited from the AFLNet but enables public access to the shared memory recording code footprints amongst three major components. This also pertains to the instrumentation of the following protocol forms.

For Type-II: We reuse the AFLNet module but upgrade the underlying emulation platform Qemu to support more protocol applications. As a supplement, we develop a new environment based on the Intel Pin [23] to back the native execution of protocols in the IA-32 and x86-64 ISAs.

For Type-III: This kind of protocol application needs ad-hoc system environment support and thus system-wide instrumentation as studied in [10], [12], [20], [24], [25]. The system mode of Qemu is often referenced in practice. However, fuzzing simulated targets at the system level is often heavy and slow. As a shortcut, we borrow the environment from FirmAFL and manage to incorporate it into the FuzzFabric to prove its scalability.

By providing multiple instrumenting environments, we anticipate that polymorphic protocol software can be directly subjected to fuzzing using FuzzFabric. Furthermore, developers have the option to customize their specialized tools by adhering to the interfaces of the ProtoAgent.

IV. EXPERIMENTS AND EVALUATION

In this section, we evaluate FuzzFabric by testing 11 protocol applications in 3 forms and using 4 instrumentation methods. This evaluation aims to assess the capability of FuzzFabric in fuzzing polymorphic protocol software. The experiments were conducted on a server equipped with an Intel(R) Xeon(R) Silver 4216 CPU featuring 64 logical cores running at 2.10GHz, running Ubuntu 16.04 LTS, and possessing 94GB of main memory.

Benchmark: We select 11 real-world protocol applications as benchmarks covering protocols of DNS, SMTP, FTP, DAAP from Profuzzbench, and HTTP. For these open-source software, we could fit them into three recommended forms as needed. Their specifications are listed in Table II and Table V.

TABLE II
SPECIFICATION OF BENCHMARKS PROTOCOL APPLICATIONS

Name	Protocol	Version	Size
BFTPD [26]	FTP	v5.7	181K
DNSmasq [27]	DNS	v2.73	1.5M
Forked-daapd [28]	DAAP	v27.2	3.0M
Exim [29]	SMTP	v4.89	3.5M
Lighttpd [30]	HTTP	v1.4.74	1.8M

Experiment configurations: For the performance evaluation, we have selected AFLNet as the baseline for Type-I and FirmAFL for Type-III. Additionally, we have chosen both standard metrics to gauge grey-box fuzzing techniques and unique metrics that align with the specific objectives of our research. The selected metrics are outlined below:

- **Line Coverage:** the common index used by grey-box fuzzing to show its capability of code exploration.

- **Throughput:** the index to show the fuzzing speed given our loose-couple design and the introduction of additional layers. About the speed variations due to the factor of protocol states, this is measured as the average number of test cases completed in one second.

The initial seed corpus was selected from the ProFuzzBench’s datasets. To minimize the influence of randomness, we conducted each 24-hour fuzzing campaign 3 times.

Performance on Type-I protocols: As AFLNet only supports llvm-instrumented protocol applications because of the emulation issues from low version Qemu, Type-I is the only group we can select to compare directly with it. The results are demonstrated in Table III covering all recommended metrics. Generally, the FuzzFabric functions effectively across all protocols, delivering improvements in both line coverage and throughput. Detailedly, forked-daapd gains the most prominent 3.7x speed up, which in turn brings 1.6% more line coverage. The worst acceleration is from BFTPD but the line coverage is increased by 1.5%. Although the fuzzing engine and the SUT are isolated and communicate with each other through the ProtoAgent, our optimizations in the fuzzing procedure can compensate for this penalty and even bring speed acceleration. The trivial improvements of line-coverage largely attributed to the seeding methods that we share with AFLNet. Unless they are further optimized, line-coverage will not benefit too much from higher throughput, for example, 1.9x speed up yields no line-coverage increment in Exim.

TABLE III
PERFORMANCE COMPARISON ON TOP OF TYPE-I PROTOCOL SOFTWARE

Subjects	Instrumentation tools	Line-coverage(%)	Throughput (executions/sec)
BFTPD	AFLnet	21.9	8.1
	FuzzFabric	23.4(+1.5)	8.5 (1.0×)
DNSmasq	AFLnet	12.2	28.9
	FuzzFabric	12.8(+0.6)	64.4 (2.2×)
Forked-daapd	AFLnet	6.7	0.9
	FuzzFabric	8.3(+1.6)	3.3 (3.7×)
Exim	AFLnet	13	3.4
	FuzzFabric	13	6.3 (1.9×)
Lighttpd	AFLnet	13.3	21.8
	FuzzFabric	13.47(+0.17)	22.4 (1.0×)

Performance on Type-II protocols: For protocol software of this type, we primarily show the results from two different instrumentation tools, i.e., Pin and user-mode Qemu, of the FuzzFabric. Assuming these target protocol applications are not open-source, we compile them into binaries of the x86 instruction set. Table IV firstly proves the competence of the FuzzFabric upon Type-II protocols and secondly reveals the performance difference raised from instrumentation methods. For example, BFTPD achieves higher efficiency using Pin while Exim using Qemu. This indicates that it is meaningful to offer or to be handy to connect to diverse instrumentation tools to help fuzzing practices better.

Performance on Type-III protocols: Based on FirmAFL, we implement the system-mode instrumentation environment

TABLE IV
PERFORMANCE COMPARISON ON TOP OF TYPE-II PROTOCOL SOFTWARE BETWEEN TWO INSTRUMENTATION TOOLS

Subjects	Instrumentation tools	Line-coverage(%)	Throughput (executions/sec)
BFTPD	Pin	23.3	45.1
	Qemu-user	23	13.5
DNSmasq	Pin	13.1	64.0
	Qemu-user	12.8	64.4
Forked-daapd	Pin	7.6	4.8
	Qemu-user	8.3	4.3
Exim	Pin	12.2	2.3
	Qemu-user	12.8	4.6
Lighttpd	Pin	13.5	27.4
	Qemu-user	13.5	26.5

for Type-III protocol software. In this experiment, we fuzz the embedded HTTP protocol through full-system simulation of 6 IoT firmwares, which can run normally according to the source code of FirmAFL in the repository [31]. The specifications of 6 *httpd* applications are listed in Table V.

Because fuzzing firmware cannot calculate line coverage, we use the number of paths discovered as a code coverage indicator. As shown in Table VI, in contrast to FirmAFL, Fuzzfabric introduces an average of 0.46% loss in paths and 6.2% throughout but gets almost the same number of crashes. After checking the FirmAFL implementation, we find our fuzzing procedure optimizations are redundant and this in addition to the introduction of the ProtoAgent inversely drag down the overall fuzzing speed. But this also impacts the discovered paths slightly as analyzed previously.

Generally, the fact that the FuzzFabric finds almost as many crashes as FirmAFL does proves its effectiveness in bug finding and its scalability of integrating a full-system instrumentation environment for Type-III protocols.

TABLE V
SPECIFICATION OF 6 HTTP PROTOCOL APPLICATIONS IN IOT FIRMWARES

Model	Vendor	Version	Program	Size
DIR-815	Dlink	1.01	httpd	90.2K
DIR-825	Dlink	2.02	httpd	444.7K
DIR-850L	Dlink	1.03	httpd	82.6K
DIR-817LW	Dlink	1.02B01	httpd	82.4K
DAP-2695	Dlink	1.11.RC044	httpd	166.1K
TEW-632	TRENDnet	1.010B32	httpd	320.9K

Need for scalability: In our experiment, the HTTP protocol is unique due to its 3 distinct forms and 4 instrumentation methods. It serves as an exemplar for justifying the need to develop a scalable fuzzing framework for two main reasons. Firstly, the HTTP protocol, even when the same, requires different forms to adapt to the underlying execution environment. Secondly, various protocol implementations consistently customize protocols to extend the functionality of standard HTTP.

TABLE VI
PERFORMANCE COMPARISON ON TOP OF TYPE-III PROTOCOL SOFTWARE
IN IOT FIRMWARES

Model	Indicators	FirmAFL	FuzzFabric
DIR-815	paths	315.7	347.0
	crashes	56	55
	throughout	370.5	348.7
DIR-825	paths	154.7	156.7
	crashes	0	0
	throughout	106.9	91.4
DIR-850L	paths	507.0	532.7
	crashes	20	22
	throughout	337	315.9
DIR-817LW	paths	293.0	292.0
	crashes	67	64
	throughout	291.7	280.9
DAP-2695	paths	912.0	786.3
	crashes	14	19
	throughout	265.6	234.5
TEW-632	paths	240.3	226.0
	crashes	0	0
	throughout	210.76	206.12

This scenario is widespread, e.g., the Apache HTTP being extensively modified and applied in various IoT firmware.

Consequently, the fuzz testing of customized protocols also needs to consider scalability, as evidenced in Table IV. Nonetheless, these tools are crucial factors for a fuzzing framework. Creating a scalable framework that facilitates the integration of various instrumentation tools provides flexibility in selecting fuzzing protocols. This allows for leveraging the best available tools based on performance requirements, or tailoring dedicated tools to address capability and/or performance issues. Additionally, this approach aids in validating and enhancing fuzzing techniques.

V. RELATED WORK

The conventional grey box fuzzing approaches primarily aim to introduce new techniques and tools for fuzzing network protocol software. Pham [7] extended AFL based on the protocol software features and developed AFLNet, which selects and mutates seeds to trigger new paths or states, leveraging coverage feedback and protocol state information to guide testing towards achieving better code coverage and protocol state coverage. Building upon AFL, Natella [8] employed fine-grained compile-time instrumentation to gather runtime information for protocol state inference and implemented StateAFL. Additionally, new network protocol fuzzing approaches such as SGfuzz [32], SnapFuzz [9], and SNPS-Fuzzer [11] aims to enhance performance by achieving more software states, constructing system-emulation environments, or capturing the entire runtime information, specifically tailored for special protocols. However, these programs are limited to operating on either the source or binary protocol software and do not adapt well to various types of protocols.

Avatar2 [33] enables analysts to select various execution environments and collectively conduct their analysis tasks on

the same execution state. The analysis state acquired by different tools can be shared across multiple analysis environments. Sebastian Österlund [25] introduces CollabFuzz, a collaborative fuzzing framework that facilitates the cooperation of multiple fuzzers under an informed scheduling policy based on central analyses. It emphasizes fuzzer collaboration. Overall, contemporary prominent fuzzing frameworks do not prioritize scalability for fuzzing, particularly for the polymorphic protocol software of network service.

VI. CONCLUSION

In view of the limitations of current grey-box fuzzing frameworks in targeting polymorphic protocol services, we advocate for the separation of the fuzzing engine from the tested targets and the development of a modularized and scalable framework to accommodate diverse forms of protocol software effectively. Assessments conducted on real-world protocol applications of varying forms substantiate the effectiveness of our proposed approach. We consider scalability to be a crucial aspect in network protocol fuzzing, and a scalable and user-friendly fuzzing framework significantly aids in fuzzing endeavors. This will further advance our pursuit of a more adaptable FuzzFabric.

VII. ACKNOWLEDGMENT

The work was supported by the Natural Science Foundation of China (61902416, 62302050) and Natural Science Foundation of Hunan Province in China (2019JJ50729).

REFERENCES

- [1] Microsoft, "Http protocol stack remote code execution vulnerability," Website, <https://msrc.microsoft.com/update-guide/vulnerability/CVE-2022-21907>, 2023-11-19.
- [2] R. Kaksonen, M. Laakso, and A. Takanen, "System security assessment through specification mutations and fault injection," in *Proceedings of the IFIP TC6/TC11 International Conference on Communications and Multimedia Security Issues of the New Century*, 2001, p. 27.
- [3] G. Banks, M. Cova, V. Felmetger, K. C. Almeroth, R. A. Kemmerer, and G. Vigna, "Snooze: Toward a stateful network protocol fuzzer," in *Information Security, 9th International Conference, ISC 2006*, pp. 343–358.
- [4] "boofuzz: Network protocol fuzzing for humans," Website, <https://boofuzz.readthedocs.io/en/stable/index.html>, 2023-11-19.
- [5] "Openrce/sulley: A pure-python fully automated and unattended fuzzing framework," Website, <https://github.com/OpenRCE/sulley>, 2023-11-19.
- [6] H. Gascon, C. Wressnegger, F. Yamaguchi, D. Arp, and K. Rieck, "Pulsar: Stateful black-box fuzzing of proprietary network protocols," in *Security and Privacy in Communication Networks*, B. Thuraisingham, X. Wang, and V. Yegneswaran, Eds., 2015, pp. 330–347.
- [7] V.-T. Pham, M. Böhme, and A. Roychoudhury, "Aflnet: A greybox fuzzer for network protocols," in *13th IEEE International Conference on Software Testing, Validation and Verification*, 2020, pp. 460–465.
- [8] R. Natella, "Stateafl: Greybox fuzzing for stateful network servers," *Empirical Software Engineering*, vol. 27, no. 7, p. 191, 2022.
- [9] A. Andronidis and C. Cadar, "Snapfuzz: high-throughput fuzzing of network applications," in *ISSTA '22: 31st ACM SIGSOFT International Symposium on Software Testing and Analysis*, 2022, pp. 340–351.
- [10] Y. Zheng, A. Davanian, H. Yin, C. Song, H. Zhu, and L. Sun, "Firm-afl: High-throughput greybox fuzzing of iot firmware via augmented process emulation," in *28th USENIX Security Symposium, USENIX Security 2019*. USENIX Association, 2019, pp. 1099–1114.
- [11] J. Li, S. Li, G. Sun, T. C. 0002, and H. Yu, "Snpsfuzzer: A fast greybox fuzzer for stateful network protocols using snapshots," *IEEE Transactions on Information Forensics and Security*, vol. 17, pp. 2673–2687, 2022.

- [12] Z. Gao, W. Dong, R. Chang, and Y. W. 0002, "Fw-fuzz: A code coverage-guided fuzzing framework for network protocols on firmware," *Concurrency - Practice and Experience*, vol. 34, no. 16, 2022.
- [13] J. Krupp, I. Grishchenko, and C. Rossow, "AmpFuzz: Fuzzing for amplification DDoS vulnerabilities," in *31st USENIX Security Symposium (USENIX Security 22)*. USENIX Association, 2022, pp. 1043–1060. [Online]. Available: <https://www.usenix.org/conference/usenixsecurity22/presentation/krupp>
- [14] X. Zhang, C. Zhang, X. Li, Z. Du, Y. Li, Y. Zheng, Y. Li, B. Mao, Y. Liu, and R. H. Deng, "A survey of protocol fuzzing," 2024.
- [15] M. Zalewski, "american fuzzy lop," Website, <https://lcamtuf.coredump.cx/afl/>, 2017-05-13.
- [16] F. Bellard, "Qemu, a fast and portable dynamic translator," in *Proceedings of the FREENIX Track: 2005 USENIX Annual Technical Conference, April 10-15, 2005, Anaheim, CA, USA*. USENIX, 2005, pp. 41–46. [Online]. Available: <http://www.usenix.org/events/usenix05/tech/freenix/bellard.html>
- [17] "panda-re/panda: Platform for architecture-neutral dynamic analysis," Website, <https://github.com/panda-re/panda>, 2023-11-19.
- [18] "Gns3/dynamips: Dynamips development," Website, <https://github.com/GNS3/dynamips>, 2023-11-19.
- [19] R. Natella and V.-T. Pham, "Profuzzbench: a benchmark for stateful protocol fuzzing," in *ISSTA '21: 30th ACM SIGSOFT International Symposium on Software Testing and Analysis*, 2021, pp. 662–665.
- [20] B. Yu, P. Wang, T. Yue, and Y. Tang, "Poster: Fuzzing iot firmware via multi-stage message generation," in *Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security, CCS 2019*. ACM, 2019, pp. 2525–2527.
- [21] "aflnet/tutorials at master · aflnet/aflnet · github," Website, <https://github.com/aflnet/aflnet/tree/master/tutorials>, 2023-11-19.
- [22] "The llvm compiler infrastructure," Website, <https://llvm.org/>, 2023-11-19.
- [23] H. Patil, R. Cohn, M. Charney, R. Kapoor, A. Sun, and A. Karunanidhi, "Pinpointing representative portions of large intel® itanium® programs with dynamic instrumentation," in *37th International Symposium on Microarchitecture (MICRO-37'04)*, 2004, pp. 81–92.
- [24] C. Zhang, Y. Li, H. Chen, X. Luo, M. Li, A.-Q. Nguyen, and Y. Liu, "Biff: Practical binary fuzzing framework for programs of iot and mobile devices," in *36th IEEE/ACM International Conference on Automated Software Engineering, ASE 2021*. IEEE, 2021, pp. 1161–1165.
- [25] S. Österlund, E. Geretto, A. Jemmett, E. Güler, P. Görsz, T. Holz, C. Giuffrida, and H. Bos, "Collabfuzz: A framework for collaborative fuzzing," in *EuroSys '21: Sixteenth European Conference on Computer Systems, 2021*. ACM, 2021, pp. 1–7.
- [26] "Bftpd – a fast, secure, easy to configure ftp server," Website, <https://bftpd.sourceforge.net/>, 2024-05-17.
- [27] "Dnsmasq - network services for small networks." Website, <https://dnsmasq.org/doc.html>, 2024-05-17.
- [28] "sfeakes/forked-daapd: Linux/freebsd daap (itunes) and mpd media server with support for airplay devices (multiroom), apple remote (and compatibles), chromecast, spotify and internet radio." Website, <https://github.com/sfeakes/forked-daapd>, 2024-05-17.
- [29] "Exim internet mailer," Website, <https://www.exim.org/>, 2024-05-17.
- [30] "Home - lighttpd - fly light," Website, <https://www.lighttpd.net/>, 2024-05-17.
- [31] "zyw-200/firmafl: Firm-afl is the first high-throughput greybox fuzzer for iot firmware." Github, <https://github.com/zyw-200/FirmAFL>, 2023.
- [32] J. Ba, M. Böhme, Z. Mirzamomen, and A. Roychoudhury, "Stateful greybox fuzzing," in *31st USENIX Security Symposium, USENIX Security 2022*, K. R. B. Butler and K. Thomas, Eds. USENIX Association, 2022, pp. 3255–3272.
- [33] M. Muench, D. Nisi, A. Francillon, and D. Balzarotti, "Avatar?: A multi-target orchestration platform," in *Workshop on Binary Analysis Research (BAR)*, 2018.